

G04 · Miras — 모든 것은 연결되어 있다

G04 · Miras — 모든 것은 연결되어있다

이 권은 **Miras** 논문 (It's All Connected, arXiv:2504.13173, Google Research, 2025) 이 던진 한 문장을 시스템 엔지니어의 눈높이로 풀어낸다: “지금까지 나온 sequence 모델은 전부 같은 종류의 물건이었다.” 그 물건이 무엇이고, 어떤 **네 개의 손잡이 (knob)** 로 조립되는지, 그리고 그 손잡이를 어떻게 돌리면 우리가 아는 linear attention·DeltaNet·Titans·심지어 softmax attention 까지 한 좌표계 위의 점으로 정리되는지를 본다. 목표는 증명이 아니라 **모델링에 쓸 수 있는 지도**를 갖는 것이다.

1. 왜 이 권이 필요한가: Titans 는 지도 위의 “한점” 이었다

앞 권 (G03) 에서 본 **Titans** 는 훌륭했지만 하나의 특정 설계였다. memory 를 작은 MLP 로 두고, token 마다 gradient descent 로 그 MLP 를 학습시키고, “past surprise” 라는 관성 (momentum) 과 “forgetting” 이라는 감쇠를 붙였다. 동작은 했다. 그런데 질문이 남았다.

- 왜 하필 ℓ_2 **regression** 을 내부 목적으로 골랐나? (관행이지 이유가 아니다)
- 왜 하필 **momentum + weight decay** 인가? 다른 학습 방법은?
- “forgetting” 이라는 말은 정말 올바른 개념인가?

직관 Titans 는 “test time 에 학습하는 memory 를 만들 수 있다” 는 **존재증명**이었다. Miras 는 그 존재 증명이 열어젖힌 **공간 전체의 지도**다. Titans 가 지도 위의 한 점이라면, Miras 는 그 점을 포함하는 좌표축을 그린다.

Miras 의 답은 새 아키텍처가 아니라 **분류학 (taxonomy)** 이다. “모델을 하나 더 만드는” 대신 “모든 모델이 사는 공간의 좌표계” 를 정의한다. 이 관점이 왜 유용한가? 좌표계가 있으면, 새 모델을 만들 때 “어느 축을 어떻게 돌릴지” 만 정하면 되고, 서로 다른 모델의 serving 비용을 **같은 잣대로** 비교할 수 있기 때문이다. 시스템 엔지니어에게 이걸 카탈로그가 아니라 **모델링 도구**다.

2. 큰 그림: 모든 sequence 모델 = “읽으며 계속 학습되는 작은메모리”

Miras 의 출발점은 단순하다. 거의 모든 sequence 모델의 내부에는 **associative memory** 가 하나 있다. associative memory 란 “key 를 넣으면 value 를 돌려주는 작은 사전” 이다. 그런데 이 사전은 고정된 표 (lookup table) 가 아니라, **token 이 하나씩 들어올 때마다 조금씩 다시 학습되는 작은 모델**이다.

비유 memory 를 “읽으면서 계속 과외를 받는 학생” 이라고 보자. token t 가 오면, 그 학생에게 “이 key 를 보면 이 value 를 답해” 라고 한 문제가 가르친다 (write). 그다음 query 가 오면 학생에게 물어본다 (read). 학생은 매문제마다 아주 조금씩 똑똑해진다. context 전체가 곧 이 학생의 **과외 커리큘럼**이다.

이 “한 문제 가르치기” 를 수식으로 쓰면 online gradient descent 한걸음이다.

$$W_t = W_{t-1} - \eta_t \nabla_W \ell(W_{t-1}; k_t, v_t)$$

기호 풀이 W_t = token t 까지 학습된 memory 의 내부 weights(= 학생의 현재실력). W_{t-1} = 직전 상태. k_t, v_t = 이번 token 의 key 와 value(입력 x_t 의 linear projection, 즉 $k_t = W_K x_t, v_t = W_V x_t$). ℓ = 이 memory 가 “얼마나 틀렸나” 를 재는 내부손실 (loss). $\nabla_W \ell$ = 그 틀린 정도를 줄이는 방향 (gradient). η_t = 이번 걸음의 보폭 (learning rate).

이 식은 이런 뜻이다. “이번 token 을 보고 memory 가 틀린만큼 ($\nabla_W \ell$), 그 틀린 방향의 반대로 보폭 η_t 만큼 memory 를 고친다.” 이게 write 다.

직관 — surprise 의 정체. Titans 가 “past surprise” 라 부른 것은 여기서 그냥 $\nabla_W \ell$, 즉 “이번 token 에서 memory 가 얼마나 틀렸나 (=놀랐나)” 다. surprise 가 크다 = gradient 가 크다 = memory 를 많이 고친다. Miras 관점에서 surprise 는 특별한 개념이 아니라 어떤 손실을 골랐든 그 손실의 gradient 일 뿐이다.

여기서 결정적인 관찰 하나가 나온다. 이 학습은 두 개의 loop 로되어 있다 (이 구조는 뒤 § 7 에서 시스템 관점으로 다시 파고든다).

핵심 memory 안의 W_t 는 **test time** 에, **token** 마다 학습된다 (inner loop). 반면 key/value 를 만드는 projection 이나 gate 를 정하는 규칙은 **미리 pre-training** 으로 학습된다 (outer loop). 즉 이걸 “학습하는법을 학습하는” **meta-learning(bilevel)** 구조다.

3. 네 개의 손잡이 (four knobs)

Miras 의 핵심 주장: 위의 “작은 memory” 를 조립하는 데는 독립적인 네 개의 손잡이면 충분하고, 기존의 모든 모델은 이 네 손잡이를 특정위치에 맞춘 한 점이다.

네 손잡이를 일상어로 옮기면 이렇다.

손잡이	논문 용어	한 줄 뜻	비유
① 무엇으로 저장?	memory architecture	학생의 머리 구조 (vector? matrix? MLP?)	공책의 종류
② 무엇을 맞추는지?	attentional bias	내부 손실 ℓ — 무엇을 잘 기억할지의 기준	과외 채점 기준
③ 얼마나 기억?	retention gate	새것 배우기 vs 옛것 지킴의 균형	복습 정책
④ 어떻게 학습?	memory algorithm	GD? momentum? Newton?	공부 방법

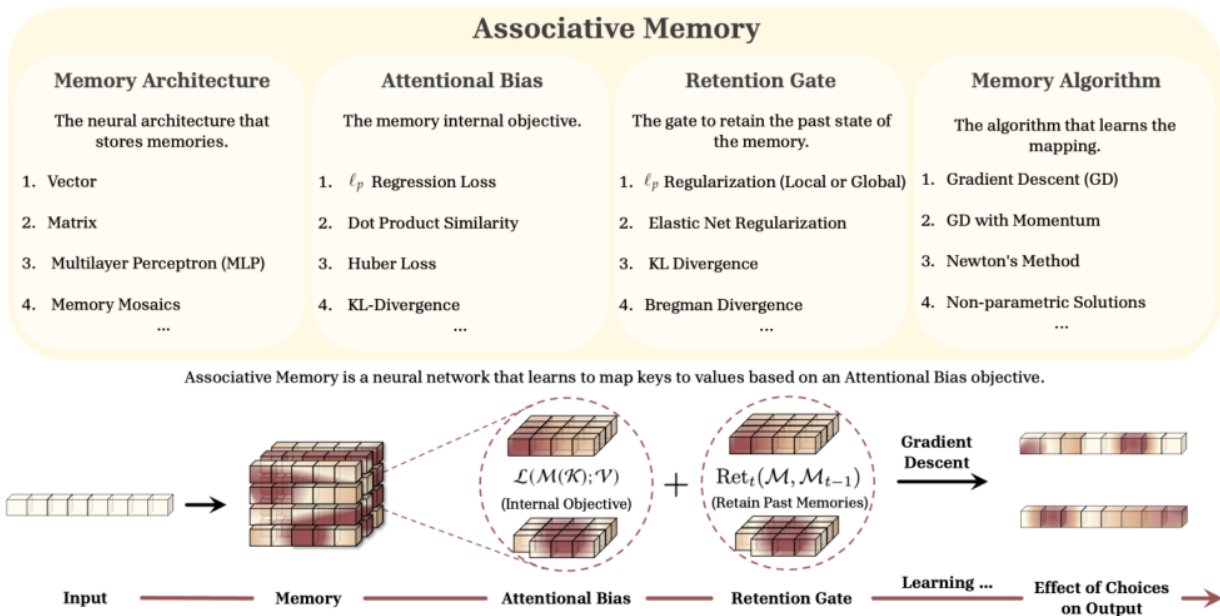


그림 G04-1 — Miras framework 의 개요. 하나의 associative memory 를 네 개의 독립 손잡이 (memory architecture / attentional bias / retention gate / memory algorithm) 로 분해한다. 매 token 최소화되는 내부목적함수는 “attentional bias(내부 목적) + retention gate(과거 유지)” 의합이고, 그것을 gradient descent 로 푼다. 출처: Behrouz et al., It’s All Connected (Miras, arXiv:2504.13173) Fig.1 — 원저자 그림.

직관 이 네 손잡이 중 앞 세대 연구가 열심히 돌린 건 ① (구조) 과 ③ (gate) 뿐이었다. Miras 는 특히 ②와 ③에 **새 선택지를 채워 넣으면** 무슨 일이벌어지는지를 실험한다. 그게 뒤에 나올 Moneta/Yaad/Memora 다.

두 손잡이는 이름값을 하므로 따로 개념을 잡고 가자.

손잡이 ② attentional bias — “무엇을 우선해서기억할까”

attentional bias 는 memory 의 **내부 목적**, 즉 ℓ 이무엇인가다. “인지과학에서 특정 자극을 우선하는 성향” 이라는 말에서 빌려온이름으로, “이 **memory** 가 어떤 사건을 우선해 기억하는가” 를 정하는손실이다.

관찰 (Miras). 지금까지 나온 거의 모든 모델의 attentional bias 는단 두 종류였다 — **dot-product similarity**(비슷한 key 에 비례해크게 써라) 아니면 ℓ_2 **regression**(key → value 를 정확히맞춰라). 이 좁은 선택을 손잡이로 인식한 순간, “다른 손실을넣으면?” 이라는 질문이 자연스러워진다.

손잡이 ③ retention gate — “forgetting 은 없다, retention 이있을뿐”

Miras 의 가장 개념적인 재정의. 기존의 “forget gate” 는 사실 memory 를지우는 게 아니다.

핵심 모델은 기억을 소거하는 게 아니라, “새 association 을 배우는것” 과 “이전 state 에 머무르는 것” 사이에서 **유지하지 않기로 결정**할 뿐이다. 뇌가 기억을 지우는 게 아니라 접근 불가능해지는 것과 같다. 그래서 Miras 는 forget gate 를 **retention gate**(유지게이트) 로 개명한다.

기호 풀이 retention 은 두 조각으로 나뉜다. **local retention** = 직전 상태 W_{t-1} 에서 얼마나 멀어지지 않을지 (이미 배운 걸 지키는 힘). 그 계수 η_t 를 논문은 **meta in-context learning rate** 라 부른다 — 크면 새것을 많이 배우고 옛것을 많이 놓는다. **global retention** = state 자체의 크기를 제한하는 힘 (고전적 gate 가 사는 곳). α_t 가 그 계수다.

비유 retention gate = 학생의 **복습 정책**. “새 문제에 집중할래 (plasticity)” vs “배운 걸 잊지 않을래 (stability)” 의 다이얼. Miras 는 long-context 성능을 좌우하는 **가장 결정적인 손잡이**가 바로 이거라고 주장한다 (뒤 § 8 의 ablation 이 이를 뒷받침).

4. 손잡이를 돌려 기존 모델을 재현하기

지도가 진짜 지도하려면, 아는 도시들이 그 위에 찍혀야 한다. Miras 는 손잡이 조합만 바꿔 기존 모델을 전부 재유도한다. 수식 유도는 생략하고 **어느 손잡이가 무엇으로 맞춰졌는지**만 본다.

Hebbian 계열 (손잡이②=dot-product). 유도하면 순수 덧셈 write 가 나온다.

$$W_t = \alpha W_{t-1} + v_t k_t^\top$$

기호 풀이 α = 옛 기억을 얼마나 남길지의 감쇠 계수 (retention). $v_t k_t^\top$ = value 와 key 의 outer product, 즉 “이 key 엔 이 value” 라는 한 장의 연관을 memory 에 **더하는** 항. ($\alpha = 1 \rightarrow$ linear attention, α 가 학습 상수 $\rightarrow$ RetNet, α_t 가 data-dependent $\rightarrow$ Mamba-2.)

이 식은 이런 뜻이다. “옛 memory 를 α 배로 살짝 흐리고, 이번 연관을 그냥 덧 쓴다.” dot-product 목적은 “고쳐쓰기” 를 요구하지 않으므로 덮어쓰기가 없다 — Hebbian 이 덧쓰기만 하는 이유가 목적 함수 수준에서 설명된다.

delta 계열 (손잡이②= ℓ_2 regression). 목적만 바꿨는데 write 가 “고쳐쓰기” 로 변한다.

$$W_t = \alpha_t W_{t-1} (I - \eta_t k_t k_t^\top) + \eta_t v_t k_t^\top$$

기호 풀이 $I =$ 항등행렬. $(I - \eta_t k_t k_t^\top) =$ “이 key 방향에 이미 저장돼 있던 옛 value 를 먼저 지우는” 보정 항. $\eta_t =$ 보폭. 나머지는 위와 같다. ($\alpha = 1 \rightarrow$ DeltaNet, $\alpha_t \rightarrow$ Gated DeltaNet, channel-wise $\rightarrow$ RWKV-7.)

이 식은 이런 뜻이다. “이 key 에 옛날에 써둔 걸 먼저 지우고 (고쳐쓰기), 새 value 를 써넣는다.” G02 에서 본 “덧쓰기 $\rightarrow$ 고쳐쓰기” 의 진화가, 여기서는 “손잡이② 를 dot-product 에서 ℓ_2 로 돌린 것” 한 문장으로 압축된다.

그리고 softmax attention 도 한 점이다. attention 은 손잡이④를 “학습하지 않고 닫힌 해로 푼다” 에 맞춘 극단이다. retention 이 없어서 과거 (k, v) 를 압축 없이 전부 보존한다 — 이게 바로 KV cache 다.

핵심 attention 의 state 가 context 길이에 비례해 선형으로 커지는 이유는, attention 이 “압축하지 않는 associative memory” 이기 때문이다. recurrent 모델들은 같은 memory 를 고정 크기로 압축한 것이다. 이한 문장이 KV cache 와 recurrent state 를 하나의 축 위에 세운다.

주의 이 표의 등호를 “구현이 완전히 같다” 로 읽으면 과독이다. 논문 각주 스스로 인정하듯, retention 열의 “ ℓ_2 ” 는 세부가 다른 gate 들을 뭉뚱그린 것이다. 이건 분류학이지 코드 수준의 등가가 아니다.

5. 손잡이 ②를 새로 돌리기: ℓ_p 와 Huber

Miras 가 실제로 채워 넣은 새 선택지들. 공통 동기는 “ ℓ_2 는 노이즈에 약하다” 이다. ℓ_2 는 큰 오차 (residual) 를 제곱으로 벌하므로, 이상한 token 하나가 memory 를 심하게 흔든다.

ℓ_p bias. 오차를 p 제곱으로 벌한다: $L = \|\mathcal{M}(k_t; W) - v_t\|_p^p$.

기호 풀이 $\mathcal{M}(k_t; W) =$ 현재 memory 가 key k_t 에 내놓는 예측 value. $v_t =$ 정답 value. 그 차이가 residual. $p =$ 오차 민감도 다이얼 ($p \geq 1$).

직관 — p 는 “놀람에 얼마나 민감할지” 다이얼. $p < 2$ 면 큰오차의 영향을 늘려서 노이즈에 강건해진다. $p > 2$ 면 큰 오차 — 즉 심하게 놀란 token — 를 증폭해서 기억한다. 극단인 $p = 1$ 에서는 오차의 크기를 버리고 부호만 저장한다. 논문은 이를 “어떤 key 가 왔었다는 사실만 저장하고 value 크기는 저장하지 않는” value-less memory 라 부른다 — 사람이 극단적 사건의 세부를 눌러 기억하는 방어기제의 비유다.

주의 — 미분 가능성이라는 시스템 제약. $p = 1$ 의 부호 함수 Sign 과 절댓값 $|\cdot|$ 은 미분이 안 되거나 도함수가 0 이라, 그대로 두면 outer-loop backprop 이 죽는다. 그래서 $\text{Sign}(x) \approx \tanh(\nu x)$, $|x| \approx \sqrt{x^2 + \epsilon}$ 로 매끈하게 바꾼다 ($\nu =$ 매끄러움계수, $\epsilon = 10^{-6}$). 이게 왜 사활적인지는 § 7 에서 명확해진다. inference-only 세계엔 없는, 이 계열 고유의 설계 제약이다.

Huber bias. ℓ_2 와 ℓ_1 을 token 마다 골라 쓴다.

직관 residual 이 작으면 (정상 token) ℓ_2 처럼 부드럽게 배우고, residual 이 크면 (outlier·노이즈 token) ℓ_1 처럼 크기를 깎아 배운다. 사실상 **per-token gradient clipping**이다. 핵심은 그 경계 δ_t 가 **입력마다 학습된다**는 것 — 무엇이 outlier인지 memory 스스로 판단한다.

6. 손잡이 ③를 새로 돌리기 + 세신제품

retention 손잡이에 새 선택지를 넣는 것이 이 논문에서 가장 독창적인부분이다. 개념만 두 개 잡자.

직관 — soft forgetting vs hard forgetting. elastic net($=\ell_1 + \ell_2$ 결합 벌점)을 retention에 넣으면 두 종류의 잊기가 나온다. **soft forgetting** = 모든 값을 조금씩 곱해서 흐리기 (고전적 gate). **hard forgetting** = 임계값보다 작은 entry를 **정확히 0으로 스냅**해서 용량을 비우기 (soft-thresholding). 후자는 state를 sparse하게 만든다 — 압축·양자화의 자연스러운 흑이다.

직관 — retention-as-renormalization. 다른 방법: state를 **확률 분포**처럼 (합=1, 모두 양수) 강제하고, 매 step **softmax**로 **재정규화**한다. 그러면 잊기는 감쇠가 아니라 **확률 질량의 경쟁**에서 나온다. 새 기억이 커지면 다른 기억의 몫이 자동으로 줄어든다. 놀라운 시스템적 성질: **state가 context 길이와 무관하게 절대 발산할 수 없다.**

이 손잡이 조합으로 Miras는 세 모델을 출하한다. 셋 다 memory는 **2-layer MLP**(deep memory), 학습법은 **plain GD**(momentum 없음, Titans에서 의도적으로 후퇴 — expressivity를 optimizer가 아니라 목적·retention에 싣는 실험)로 고정하고, ②·③만 바꾼다.

모델	손잡이② (bias)	손잡이③ (retention)	한 줄 성격
Moneta	ℓ_p ($p=3$)	ℓ_q -norm 구속 ($q=4$) + ℓ_2	놀란 token을 날카롭게 기억 + 강한 norm 통제
Yaad	Huber (mixture)	Titans 식 local+global ℓ_2	outlier에 강건한 균형형
Memora	ℓ_2	KL / simplex (softmax 재정규화)	state가 증명 가능하게 유계

세 모델의 update를 개념 수준으로만 보면:

$$\text{Moneta: } A_t = \alpha_t A_{t-1} - \eta_t \nabla_W \ell_p, \quad W_t = \frac{A_t}{\|A_t\|_q^{q-2}}$$

기호 풀이 $A_t = \text{accumulator}$, 날것의 gradient 를 쌓아두는 두 번째 내부 state. W_t = 실제로 읽을 때 노출되는 memory. $\|A_t\|_q = A_t$ 의 q -norm(크기). α_t, η_t = channel 별로 학습되는 retention·보폭.

이 식은 이런 뜻이다. “gradient 를 accumulator 에 쌓되 (위 줄), 노출되는 memory 는 항상 norm 이 통제된 껍질 위에 있게 정규화한다 (아래줄).” Moneta 는 **state** 를 두 벌 (A_t 와 W_t) 들고 다닌다 — 뒤 § 7 의 비용 계산에서 중요하다.

주의 — W_t 에 왜 W_{t-1} 이 없나? recurrence(과거→현재연결) 는 W 가 아니라 **accumulator** A 에 있다 (A_t 가 A_{t-1} 을 쓴다). W_t 는 매 step A_t 를 norm 으로 정규화해다시 뽑는 **파생값**이라 W_{t-1} 을 직접 안 쓴다. 그렇다고 W_{t-1} 이 사라진 건 아니다 — gradient $\nabla_{\ell_p}(W_{t-1}; \cdot)$ 안에 들어간다. 진짜 recurrent state 는 A 하나이고, “두 벌” 은 A + 그 파생 W 다. (Titans 의 ℓ_2 감쇠는 W -공간에서 곱셈으로 끝나 A 가 필요 없지만, 일반 ℓ_q 유지는 “쌓기 $A \rightarrow$ 투영 W ” 의 이중 구조가 필요하다. $q=2$ 면 Titans 형태로 붕괴.)

$$\text{Yaad (Huber): } W_t = W_{t-1} - \begin{cases} \eta_t \nabla_W \ell_2 & \text{if } \|\mathcal{M}(k_t) - v_t\| \leq \delta_t \\ \eta_t \delta_t \nabla_W \ell_1 & \text{그 외} \end{cases}$$

이 식은 이런 뜻이다. 오차가 작으면 ($\leq \delta_t$) 부드러운 ℓ_2 gradient 로, 크면 ℓ_1 gradient 에 δ_t 를 곱해 (= gradient clipping) 갱신한다. 즉 Huber = “작은 오차엔 민감, 큰오차 (outlier) 엔 둔감” — 엄청 놀라운 토큰 하나에 메모리가 과잉 반응하지않게 하는 **coping mechanism**(outlier robust). Moneta 와 달리 Yaad 는 W_{t-1} 을 직접 쓴다 (dual accumulator 없음). δ_t = channel 별로 학습되는 Huber 임계값.

$$\text{Memora: } W_t = \text{softmax}(\alpha_t \log W_{t-1} - \eta_t \nabla_W \ell_2)$$

이 식은 이런 뜻이다. “옛 memory 에 로그를 씌워 새 gradient 를더한 뒤, softmax 로 도로 확률 분포로 만든다.” softmax 가 매 step state 를 양수·합-정규화 상태로 되돌리므로 **폭발이 원천 차단**된다.

비유 그림 오른쪽에서 $\eta \cdot \alpha$ 가 별도의 작은 (low-rank) 가지에서 뽑히는 게 보인다. 이 건 “복습 정책을 정하는 관제탑”이다. 이 관제탑 자체는 pre-training 에서 학습되고, inference 때는 **token** 마다게이트 값을 계산해 내놓기만 한다 (스스로 더 학습하진 않는다). 다음 절의 핵심 그림이다.

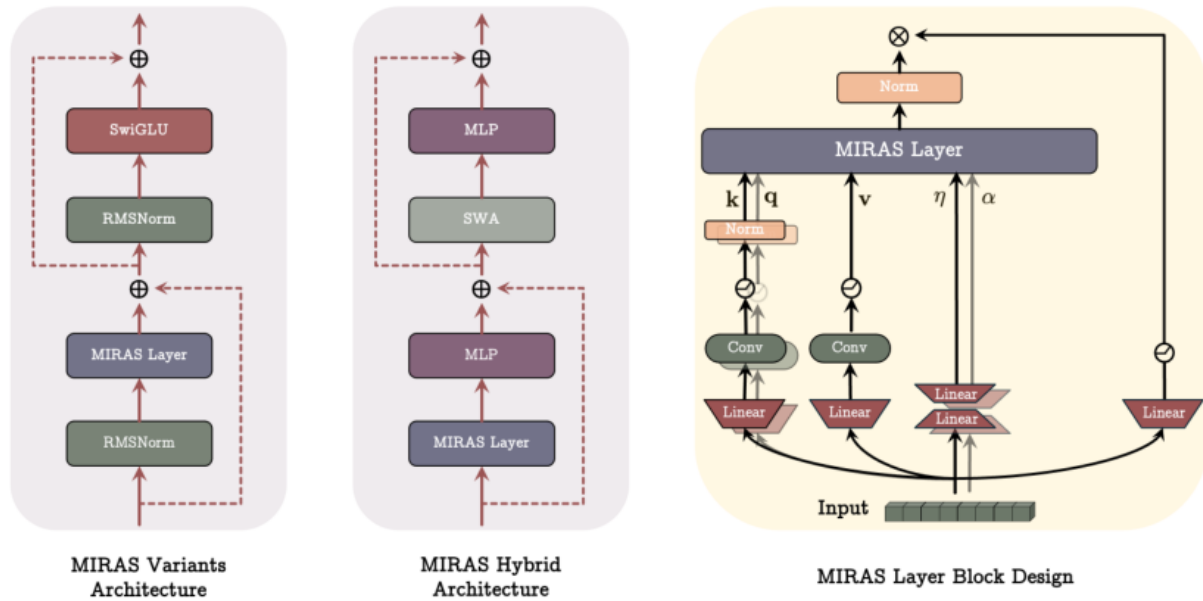


그림 G04-2 — Miras 변형의 아키텍처. (왼쪽) 순수 recurrent 블록 (RMSNorm → Miras layer → SwiGLU), (가운데) Miras layer와 Sliding Window Attention 을 번갈아 쌓는 hybrid, (오른쪽) layer 내부: k/q/v projection 뒤 depthwise conv, q·k normalization, low-rank 로 뽑히는 $\eta \cdot \alpha$ 게이트, 출력 normalization + linear gate. 출처: Behrouz et al., It's All Connected (Miras, arXiv:2504.13173) Fig.2 — 원저자 그림.

핵심 — Miras의 gate는 scalar가 아니라 channel 별 벡터다. Titans(G03)의 $\text{gate}(\theta, \eta, \alpha)$ 는 토큰당 스칼라였다. Miras 는 이를 **channel-wise** 벡터 ($\eta_t, \delta_t, \alpha_t \in \mathbb{R}^d$) 로 일반화한다 — memory의 d 개 채널이 각자 자기 유지·보존을 가진다. 그림 오른쪽의 “autoencoder 처럼 생긴 matmul 2 개” 가 바로 이것: **low-rank projection** ($d \rightarrow r \rightarrow d$, 병목 r) 으로 d 차원 gate 벡터를 싸게 뽑는다 (전체 $d \times d$ 대신 $d \times r + r \times d$). 그래서 gate 값이 스칼라가 아니라 벡터이고, Titans 보다 표현력이 크다 (채널마다 다르게 유지·기억).

7. 두 개의 loop: 훈련 vs 테스트타임

수식이 두 loop 를 오가서 헛갈리기 쉬운 지점이다. 표 하나로 가른다.

	outer loop (pre-training)	inner loop (test time)
무엇이 학습되나	projection(W_K, W_V, W_Q), conv, output gate, 게이트를 만드는 규칙, 초기 memory W_{init}	memory weights W_t (2-layer MLP), Moneta 는 A_t 까지
목적	next-token prediction	각 layer 의 attentional bias ℓ
언제 갱신	training step 마다 (inference 에션 동결)	매 token

	outer loop (pre-training)	inner loop (test time)
어느 보폭	outer η (사람이 정하는 스케줄)	inner η_t (token 마다 자동 생성)

핵심 — “게이트는 누가 학습하나?” 게이트값 ($\eta_t, \alpha_t, \delta_t$) 은 inner loop 에서 소비되지만, 게이트를 만드는 정책은 **outer loop** 가 학습한다. outer loop 는 “inner loop 가 얼마나 공격적으로 쓰고 얼마나 유지할지” 의 per-token·per-channel 정책을 배운다. inference 때 게이트는 token 마다 달라지지만 **학습되고 있진 않다** — 동결된 projection 이 계산해내놓는 값일 뿐이다.

주의 — **smooth surrogate** 가 사활적인 이유. outer loop 의 gradient 는 inner update 의 궤적 전체를 거슬러 흐른다 (MAML 류 “backprop through an optimizer”). 그래서 inner step 의 모든 연산이 미분가능해야 한다. § 5 의 Sign·절댓값·soft-threshold 는 꺾인점에서 미분이 안 되므로, $\tanh \cdot \sqrt{x^2 + \epsilon} \cdot \arctan$ 이 “**inner** 의 수학을 **outer** 가 미분할 수 있게 만드는 접착제” 역할을한다. inference-only 관점에는 없는, 훈련 고유의 제약이다.

직관 — **chunkwise** 로 훈련을 빠르게. token 을 하나씩 순차 처리하면 GPU 가 논다. 그래서 sequence 를 크기 C (16 또는 64) 의 chunk 로 잘라, chunk 안 모든 token 의 gradient 를 직전 **chunk** 끝 **state** 에서한꺼번에 평가한다. 그러면 chunk 내부가 **batched GEMM** 하나 + **element-wise** 후처리로 붕괴하고, 순차 의존은 chunk 경계 (L/C 개) 에만 남는다. 단 이걸 per-token 정의의 근사다 — gradient 가 최대 $C - 1$ token 묶은 state 에서 평가되고, softmax 같은 비선형은 chunk 당 한번만 발화한다 (Memora 는 이 때문에 chunk 첫 token 에서만 진짜 비선형 step 을 밟는 “lag token” 트릭을 쓴다).

8. 실험이 말하는 것 (요점만)

- 언어모델링. flagship 1.3B/100B tokens 에서 순수 recurrent 인 Yaad (ppl 15.18) 와 Moneta (15.52) 가, attention 을 섞은 hybrid Samba (16.13)·GDN-H2 (15.91) 까지 이긴다. “더 나은 attentional bias + retention 이면 attention 없이 hybrid 를 이긴다” 가 headline 주장이고, 이스케일·이 벤치마크에선 수치가 그걸 지지한다.
- long-context (S-NIAH). needle-in-haystack 에서 세 변형 평균 92~93 점 vs GDN 75.8, DeltaNet 57.9. 노이즈 haystack 에서 Moneta 가 8K 에서도 98.8 유지 — 노이즈에 강건한 p -norm 의 효과로 귀속.
- ablation 이 주장을 뒷받침. Yaad 구성요소 제거 실험에서 기여순위가 **retention gate** (− 3.35) > **deep memory** (− 2.41) > **threshold** 입력 의존성 (− 1.79) > **bias** 세부. 즉 “retention 이 결정적 레버” 라는 주장 그대로다. 또 q (retention 손잡이) 는 scaling 패턴 자체를 바꾸지만 p (bias 손잡이) 는 성능만 바꾸고 모양은 못 바꾼다.

한계 실증은 1.3B / 100B tokens 에서 끝난다. 7B+, SFT/RLHF 이후, exact-copy recall 이 지배하는 실무 부하에서도 순위가 유지되는지는미검증이다. long-context 검증도 single-needle 하나뿐 — multi-needle·multi-hop·32K 초과는 미검증이다. 그리고 wall-clock·throughput 수치가 전혀 없다. 효율 증거는 FLOPs·매칭 perplexity 곡선 하나다.

9. 시스템 모델링 관점 (이 권의핵심)

이제 이 손잡이 조합이 **decode 비용 · state 크기 · kernel** 로 어떻게번역되는지 본다. 논문은 이 산수를 하지 않으므로 아래 정량은 이 책의계산이다.

시스템 모델링 관점 — state 크기. Miras layer 의 recurrent state 는 head 당 2-layer MLP 전체다: $W_1 \in \mathbb{R}^{d \times 4d}$, $W_2 \in \mathbb{R}^{4d \times d}$, 합계 $8d^2$ 개의 부동소수 스칼라 (d = head 차원). 이걸 matrix-state 모델 (DeltaNet/GDN) 의 d^2 의 8 배이고, Moneta 는 accumulator A_t 까지 들어 16 배 ($16d^2$) 다. $d = 64$ 면 head 당 32K entry(Moneta 64K) vs matrix state 4K. 손잡이④를 “**deep memory**” 로돌린 대가가 곧 state 세금이다. per-sequence state 상주, prefix caching 용 checkpoint, speculative branch 당 복제가 전부 이 배율로 커진다.

시스템 모델링 관점 — KV cache 와의 손익분기. KV cache 는 head 당 $2Nd$ entry(N = context 길이). recurrent state $8d^2$ 와 같아지는지점은 $8d^2 = 2Nd \rightarrow N = 4d$. $d = 64$ 면 약 256 token(Moneta 는 512) 만 넘으면 recurrent state 가 cache 보다 작고, 이후엔 context 가 아무리 길어도 상수다. 즉 짧은 context에선 KV cache 가싸고, 어느 길이 (crossover) 를 넘으면 recurrent 가 이긴다 — memory 가클수록 (state 가 똥똥할수록) crossover 가 뒤로 밀린다.

시스템 모델링 관점 — decode 는 RMW 가 지배한다. per-token 갱신은네 단계다: forward(예측 형성) $\rightarrow$ backward(2-layer MLP 관통 gradient) $\rightarrow$ retention/정규화 (element-wise) $\rightarrow$ query로 읽기. FLOP로는 head-layer 당대략 $56 \sim 64d^2$ 로 GDN 류 ($\sim 3 \sim 4d^2$ MAC) 의 8~10 배지만, 결정 변수는 FLOP 이 아니다. 매 token 마다 $8d^2$ ($16d^2$) state 전체를 read-modify-write 해야 하므로 decode 는 **memory-bandwidth-bound** 다. attention decode 가 KV cache 를 read-only 로 스트리밍하는 것과 달리, 여기는 트래픽의절반이 write 다. 이 write-back 세금이 이 계열의 decode 를규정한다.

시스템 모델링 관점 — prefill/훈련은 GEMM 골격. §7 의 chunkwise 덕에 prefill·훈련은 GLA/TTT 식 kernel 과 같은 뼈대다: chunk 당 batched GEMM + element-wise epilogue, 순차 의존은 chunk 경계 (L/C 개)에만. Miras 고유 비용: Moneta 의 $\|A_t\|_q$ 는 경계마다 $8d^2$ 전역 norm reduction(GEMM 사이에 끼는 reduction kernel), Yaad 는 두 branch 로 element-wise 약 2 배, Memora 의 softmax/log 는 parameter slice 마다. 대신 Memora 의 유계·정규화 state 는 저장밀 저장에 원리적으로우호적이다 (단 log 가 소값의 양자화 오차를 키운다).

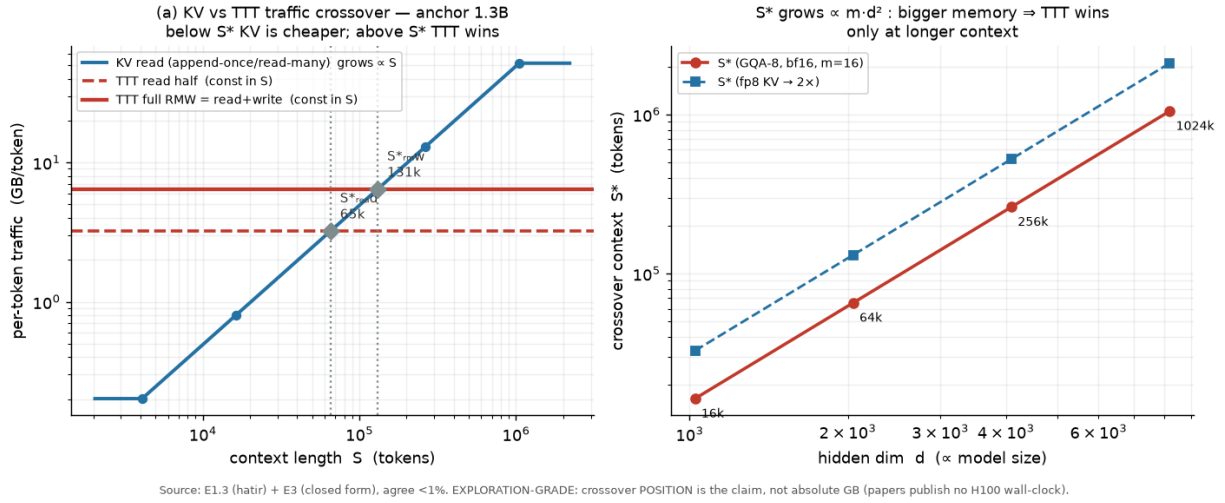
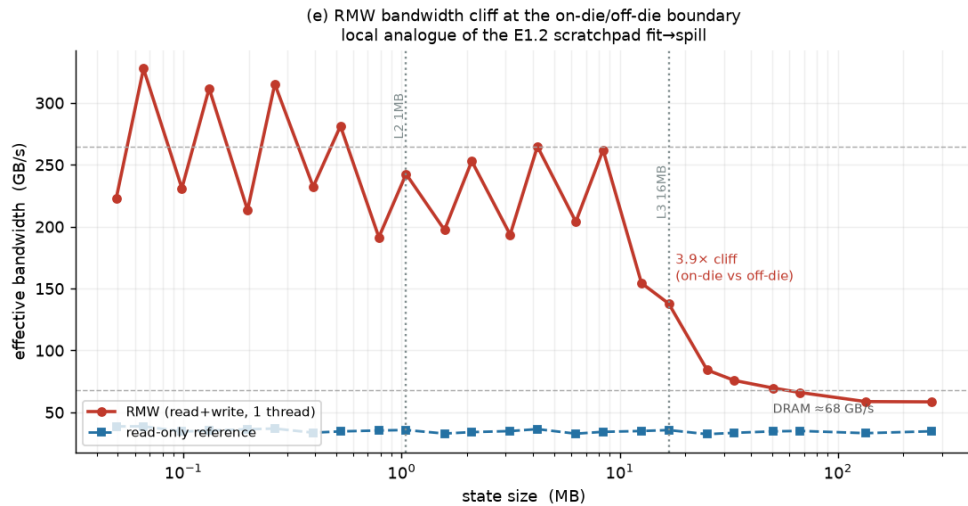


그림 G04-3 — KV cache vs recurrent(TTT) state 의 트래픽 교차점. (왼쪽) context 길이 S 가 커지면 KV read 트래픽은 선형으로 늘지만 recurrent RMW 트래픽은 상수라, 어느 지점 S 에서 교차한다. (오른쪽) 그 교차점 S 는 hidden dim d (=모델 크기) 가 클수록 뒤로 밀린다 — 큰 memory 는 더 긴 context 에서만 이득. 출처: 본서 자체 실험 (exp-a).



Source: E2.2 (CPU numpy). SHAPE-only: the 3.9× L3→DRAM cliff structure transports; GB/s values do not (H100 on/off-die ridge ~100× elsewhere). RMW ≈ 0.5× read throughput = the write-back tax KV avoids.

그림 G04-4 — read-modify-write(RMW) 의 대역폭 절벽. state 가 on-die 캐시에 들어갈 땐 대역폭이 높지만, off-die(DRAM) 로 넘어가는 순간 3.9 배 절벽이 생긴다. RMW 는 read-only 대비 약 절반의 유효 처리량 — write-back 이 그 세금이다. state 가 8~16 배 뚱뚱한 Miras 에션이 경계가 더 빨리 온다. 출처: 본서 자체 실험 (exp-e).

시스템 모델링 관점 — batching 과 hybrid. per-request fast-weight state 는 shared-weight batching 을 깨뜨린다 (모든 fast-weight 계열의 공통특성). state 가 8~16 배 커진 만큼 grouped-GEMM decode 의 per-request working set 도 그 배율. hybrid 변형 (-H) 은 SWA 를 섞으므로 window 크기의 KV cache($O(wd)$) 가 recurrent state 위에 다시엮힌다 — “cache 없는 serving” 은 순수 변형에만 해당한다.

한계 — 아까운 미실현 혹. elastic net 의 hard forgetting(0-스냅) 은 state 를 sparse 하게 만들어 압축 · 양자화의 자연스러운 혹인데, 출하된 세 모델 어디에도 실리지 않았다. serving 경제학에 가장 직접 닿는 변형이 카탈로그에만 남았다.

요약 - Miras 의 핵심은 세 모델이 아니라 좌표계다: 모든 sequence 모델= “test time 에 attentional bias 를 온라인으로 최소화하는 작은 memory”, 조립 손잡이는 네 개 (구조 / attentional bias / retention / 학습법). - **attentional bias** = 무엇을 우선 기억할지 (내부 손실). **retention** = 새것 배우기 vs 옛것 지키기 (잊기는 소거가 아니라 “유지 안 하기”). long-context 를 좌우하는 결정적 손잡이는 retention 이다. - 손잡이 ② · ③를 새로 돌린 실증이 **Moneta**($\ell_p + \ell_q$), **Yaad**(Huber+ ℓ_2), **Memora**(ℓ_2 +softmax simplex). 1.3B 에서 순수 recurrent 가 attention hybrid 까지 이겼다. - 시스템 관점: state 는 head 당 $8d^2$ (Moneta $16d^2$) — matrix state 의 8~16 배. decode 는 이 state 전체의 **RMW** 로 memory-bandwidth-bound, KV cache 와의 손익분기는 대략 $N = 4d$. wall-clock 증거는 없다.

다음 권 예고 — G05 Atlas. Miras 가 비워 둔 손잡이 ④ (학습법) 와 “per-token 단일 쌍” 목적의 한계를, 다음 권 **Atlas** 가 inner loop 에 Muon(근사 2 차 최적화) 을 이식하고 최근 window 전체를 함께 최적화하는 Omega rule 로 채우려 간다 — test-time training 이 아니라 test-time memorization 이라는 고집과 함께.